

Edge AI y Despliegue en Dispositivos

Módulo 9 · Infraestructura y Despliegue

Edge servers, movil con llama.cpp, ONNX Runtime, actualizacion OTA, Cloudflare Workers AI y casos de uso industriales.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Edge AI y Despliegue en Dispositivos

La mayoría de los sistemas LLM de empresa funcionan en modo cloud: el dispositivo del usuario envía una petición a un servidor centralizado y recibe la respuesta. Pero hay una clase creciente de casos de uso donde este modelo no funciona: latencia ultra-baja (el round-trip a la nube añade 50-200ms inaceptables), privacidad (los datos no pueden salir del dispositivo), conectividad limitada (zonas sin internet confiable), o resiliencia (el sistema debe seguir funcionando si la red falla).

Edge AI es el despliegue de modelos de IA directamente en el dispositivo del usuario o en servidores edge cercanos a la fuente de los datos, sin necesidad de conexión a la nube. En 2025, los avances en cuantización (modelos de 1-3B parámetros en Q4 GGUF caben en un iPhone 15 Pro), los chips de IA integrados en dispositivos modernos (Apple Neural Engine, Qualcomm NPU, NVIDIA Jetson), y los frameworks de inferencia móvil (llama.cpp, ONNX Runtime, MediaPipe) han hecho que el edge AI sea una opción viable para una gama mucho más amplia de aplicaciones.

El espectro del edge AI

Dispositivos móviles: smartphones y tablets

Los smartphones modernos de gama alta tienen hasta 16GB de RAM y chips de IA dedicados (Apple Neural Engine en A17/M-series, Qualcomm AI Engine). Modelos cuantizados de 1-3B parámetros en Q4 son ejecutables en dispositivos iOS y Android: Gemma 2 2B (Google), Phi-3 Mini 3.8B (Microsoft), Llama 3.2 1B/3B (Meta) están optimizados para dispositivos móviles. llama.cpp en iOS y Android es el framework estándar de inferencia local. La velocidad de generación es de 5-20 tokens por segundo, aceptable para muchos casos de uso.

Edge servers: nodos de cómputo en la red

Los edge servers son servidores en ubicaciones geográficas cercanas a los usuarios (CDN edge, plantas de manufactura, sucursales bancarias) que ejecutan modelos de mayor tamaño que los dispositivos móviles pero con menor latencia que los datacenters centrales. NVIDIA Jetson Orin (32GB GPU, 2024) puede ejecutar modelos de hasta 13B parámetros en Q4. AWS Wavelength, Azure Edge Zones, y Cloudflare Workers AI permiten desplegar inferencia en edge sin hardware propio.

El contexto de cuándo usar edge vs cloud

Edge AI es la elección correcta cuando: la latencia de red es inaceptable (<50ms de extremo a extremo), la privacidad requiere que los datos no salgan del dispositivo o de las instalaciones del cliente, la conectividad es intermitente o limitada (campo, manufactura, logística), o el volumen de datos a procesar hace que transmitirlos a la nube sea costoso o impracticable (visión artificial en líneas de producción con cámaras generando GB/hora). En todos los demás casos, cloud es más flexible y potente.

Frameworks y herramientas para edge AI

llama.cpp: inferencia eficiente en CPU/NPU

llama.cpp (Georgi Gerganov, 2023) es la implementación de referencia para inferencia de LLMs en hardware de consumo y edge. Está escrito en C++ con cero dependencias, soporta cuantización GGUF (Q2 a Q8), puede ejecutarse en CPU pura (sin GPU), y tiene backends para GPU NVIDIA (CUDA), AMD (ROCm), Metal (Apple), y Vulkan (cross-platform). Una MacBook Pro M3 Max puede ejecutar Llama 3 8B en Q4 a ~50 tokens/segundo. Ollama usa llama.cpp internamente como motor de inferencia.

ONNX Runtime y cuantización para modelos pequeños

ONNX Runtime (Microsoft) permite exportar modelos PyTorch al formato ONNX y ejecutarlos eficientemente en múltiples backends (CPU, GPU, Neural Engine de Apple, NPU de Qualcomm). Para modelos pequeños de NLP (clasificación, NER, embeddings), la conversión a ONNX con cuantización INT8 produce modelos 4x más pequeños y 2-4x más rápidos en CPU, compatibles con todos los sistemas operativos. Es la elección estándar para clasificadores y modelos de embedding en dispositivos.

Actualización y gestión de modelos en edge

Los modelos desplegados en edge necesitan actualizarse periódicamente (nuevas versiones, fine-tuning con datos recientes). Las estrategias de actualización son: over-the-air update (descargar el nuevo modelo cuando hay conexión WiFi), delta updates (solo descargar los cambios respecto al modelo anterior, posible si se usan adaptadores LoRA sobre un modelo base estático), y versioning con rollback (si el nuevo modelo tiene bugs, revertir al anterior). La gestión de modelos en edge sigue los mismos principios que el firmware update de dispositivos IoT.

SECCIÓN 4 · EJEMPLOS REALES

- Control de calidad visual en planta de manufactura (sin conectividad confiable): cámaras con NVIDIA Jetson Orin ejecutan un modelo de visión artificial fine-tuneado para detectar defectos en piezas a 30 fps. Los resultados se procesan localmente y se envían resúmenes periódicos a la nube cuando hay conectividad. La latencia de inferencia local es <100ms vs 500-2000ms con cloud. Los defectos se detectan en tiempo real sin depender de la conexión a internet.
- Apple Intelligence en dispositivos Apple: Siri con iOS 18 procesa la mayoría de las solicitudes con un modelo de lenguaje en el dispositivo (se estima ~3B parámetros, optimizado para el Apple Neural Engine). Las solicitudes más complejas se envían con Private Cloud Compute a servidores de Apple, nunca a terceros. El resultado: Siri más rápida, más privada, y con capacidades de razonamiento mejoradas.
- Asistente médico offline para zonas rurales (conectividad limitada): tabletas con Gemma 2 2B fine-tuneado sobre guías clínicas locales en Q4 GGUF. Los médicos rurales pueden consultar el asistente incluso sin conexión a internet. El modelo ejecuta a ~15 tokens/segundo en el tableta. Las actualizaciones del modelo se descargan cuando hay

conexión WiFi disponible en el centro de salud.

- Clasificación de emails en cliente de correo local (privacidad): plugin para Thunderbird/Outlook que clasifica emails con un modelo DistilBERT de clasificación en ONNX INT8 ejecutando localmente. El email nunca sale del dispositivo. La clasificación es instantánea (<50ms) sin latencia de red.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Usar modelos demasiado grandes para el dispositivo. Un modelo de 7B parámetros en Q4 requiere ~4GB de RAM. Un smartphone con 6GB de RAM dejará poco espacio para el resto de las aplicaciones, produciendo intercambio a disco y latencias de minutos. Para dispositivos móviles, modelos de 1-3B parámetros en Q4 son el límite práctico en dispositivos de gama media.

Error 2: Ignorar el consumo de batería. La inferencia de LLMs es intensiva en CPU/GPU y consume batería significativa. Un modelo que procesa 20 tokens/segundo en un smartphone puede agotar la batería en pocas horas si se usa continuamente. Diseña el sistema para minimizar las llamadas al modelo local: usar el modelo edge solo cuando sea necesario, y siempre ofrecer al usuario el control del balance rendimiento/batería.

Error 3: No planificar la actualización de modelos desde el principio. Los modelos en producción necesitan actualizarse por razones de seguridad, mejora de calidad, y adaptación a nuevos datos. Un sistema edge sin mecanismo de actualización de modelos acumula deuda técnica rápidamente. Diseña el pipeline de actualización de modelos desde el inicio del proyecto.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Para desplegar un modelo LLM en dispositivos iOS/Android con llama.cpp: exporta el modelo a GGUF con llama.cpp (./convert.py o llama-quantize), elige el nivel de cuantización apropiado para el dispositivo (Q4_K_M es el mejor balance calidad/tamaño/velocidad), integra la biblioteca de llama.cpp en tu aplicación iOS (usando Swift bindings de LLM.swift) o Android (usando llamafile o java/kotlin bindings), y gestiona el ciclo de vida del modelo (cargar en memoria cuando se necesita, descargar cuando no se usa para ahorrar batería y memoria).

Para edge servers con NVIDIA Jetson: instala JetPack SDK (que incluye CUDA, TensorRT), instala Ollama o vLLM para Jetson (requiere compilación para ARM64 con soporte CUDA de Tegra), configura el modelo con cuantización Q4 o Q5 para modelos de hasta 13B parámetros, y expone el API REST compatible con OpenAI en la red local. El Jetson Orin 64GB puede ejecutar Llama 3 13B en Q4 a ~15 tokens/segundo, suficiente para aplicaciones industriales de bajo throughput.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M4-T8 (GPUs): los chips de IA de edge (NPUs, Neural Engines) son especializaciones de la aceleración hardware para IA.
- M8-T6 (vLLM y TGI): llama.cpp es el servidor de inferencia edge, análogo a vLLM en cloud.
- M9-T1 (Cloud vs on-premise): el edge AI es el tercer modelo de despliegue junto a cloud y on-premise.

Edge AI despliega LLMs directamente en el dispositivo o en servidores edge cercanos cuando: la latencia de red es inaceptable (<50ms), los datos no pueden salir del dispositivo (privacidad), la conectividad es limitada, o el volumen de datos hace inviable transmitir a la nube. Los dispositivos móviles soportan modelos de 1-3B parámetros en Q4 (5-20 tokens/seg). Los edge servers (NVIDIA Jetson Orin 64GB) soportan hasta 13B parámetros en Q4 (~15 tokens/seg). Frameworks: llama.cpp (el estándar, soporta GGUF en CPU/GPU/NPU), ONNX Runtime (para modelos menores de clasificación), Ollama (llama.cpp con UX amigable). Sin hardware propio: Cloudflare Workers AI con latencia <50ms global.